Семинар 7. Бесконфликтно реплицируемые типы данных (CRDT)

Распределенные вычисления

Георгий Семенов

Институт прикладных компьютерных наук
Университет ИТМО

весна 2026

Организационные моменты

- (Напоминание) Второе домашнее задание – сегодня жесткий дедлайн в 23:59 02.04.2026
- Проставлю баллы по зеленым галочкам в GitHub Classroom

Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

Оптимистическая репликация

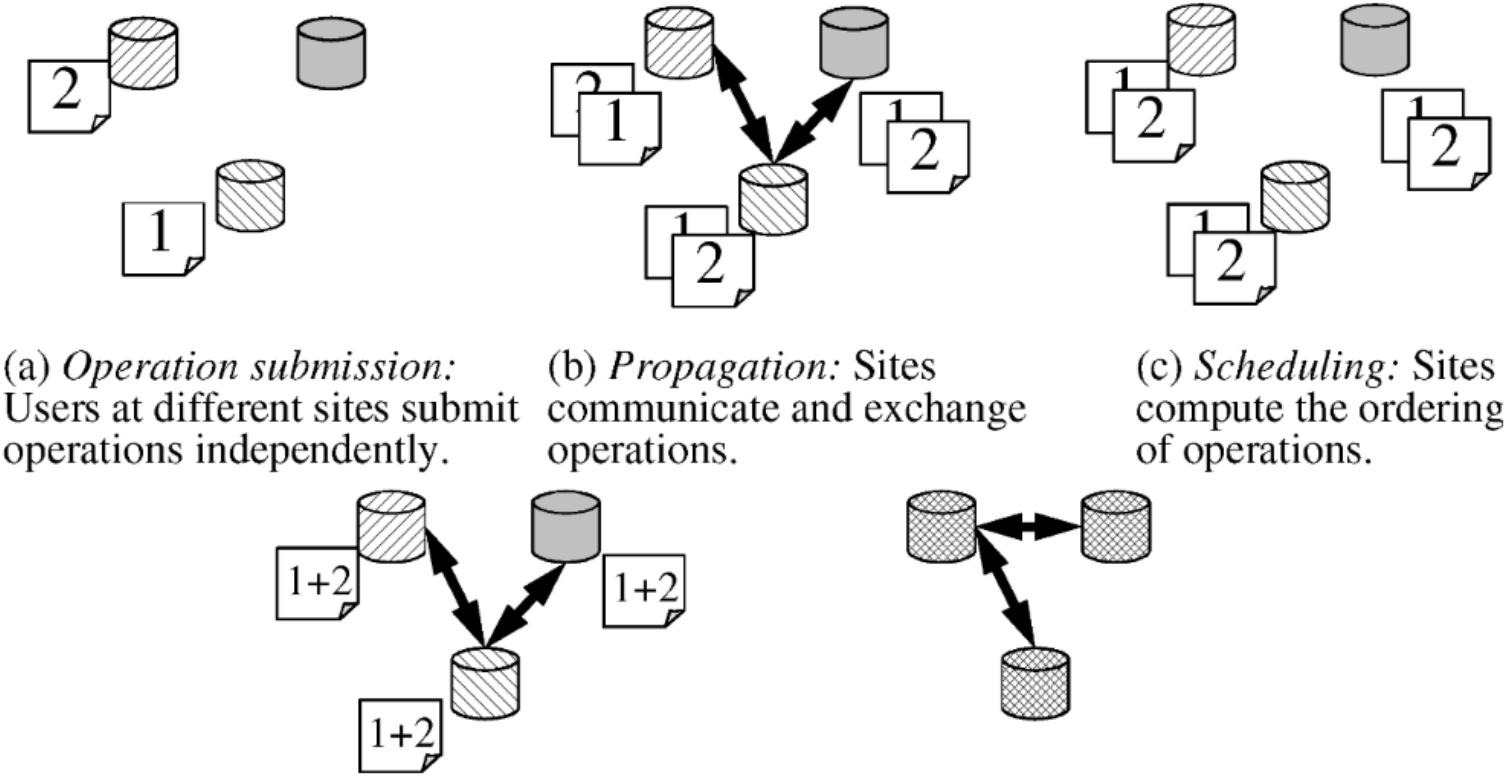
Оптимистическая репликация

Репликация, при которой мы оптимистически разрешаем write-обновления на репликах без предварительной синхронизации.

- Как это работает:
  - У каждого узла системы есть своя реплика данных
  - Чтение и запись производятся с локальной реплики, без синхронизации
  - Состояния синхронизируются асинхронно (gossip, anti-entropy)
- Как работать с конфликтами:
  - Детерминированно их разрешать без синхронизации (CRDT)
  - Полуавтоматически и централизованно (семантическая реконсиляция)
  - Полностью ручной централизованной merge

Схема оптимистической репликации

Y. Saito and M. Shapir



(a) *Operation submission:* Users at different sites submit operations independently.

(b) *Propagation:* Sites communicate and exchange operations.

(c) *Scheduling:* Sites compute the ordering of operations.

(d) *Conflict resolution:* Sites detect conflicts and transform offending operations to produce results intended by users.

(e) *Commitment:* Sites agree on the final ordering and reconciliation result. Their changes become permanent.

Fig. 1. Elements of optimistic replication and their roles. Disks represent replicas, memo sheets represent operations, and arrows represent commu-

Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

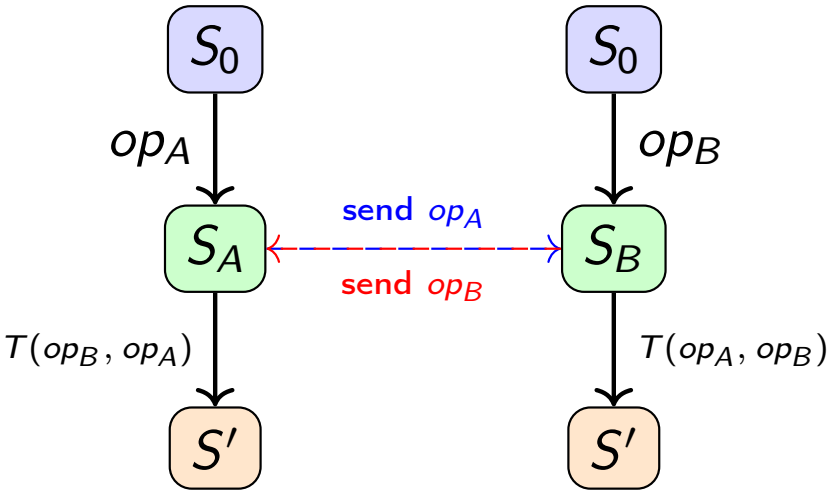
Операциональные преобразования (OT) - 1989

Ключевая идея:

- Операции отправляются как есть на все реплики
- При получении операция *преобразуется* с учётом уже применённых операций
- Функция трансформации $T(op_1, op_2) \to op_1'$

Условие корректности (TP1):
$op_1 \circ T(op_2, op_1) = op_2 \circ T(op_1, op_2)$

Системы: Jupiter (Xerox PARC, 1995), Google Wave (2009), ShareJS, Atom Teletype.



Операциональные преобразования (OT) - 1989

Проблема OT

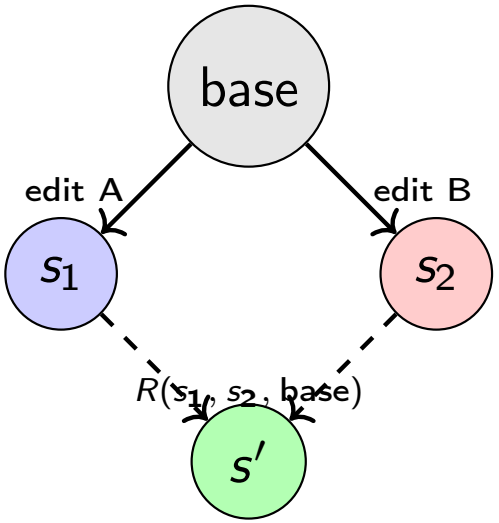
В 2000-х возникла репутация о том, что OT-алгоритмы склонны быть некорректными.
При этом – активно используется в Google Docs до сих пор.

Online-демо: operational-transformation.github.io

Семантическая реконсиляция

Marc Shapiro et al. (2000–2005), INRIA:

- Конфликты разрешаются *семантически* — система знает смысл данных
- Reconciler — функция $R(s_1, s_2, s_{\text{base}})$ с учётом общего предка
- Позволяет более умные merge (diff3, operational semantics)



Ограничения:
Требует централизованного reconciler или 3-way merge; не масштабируется тривиально.

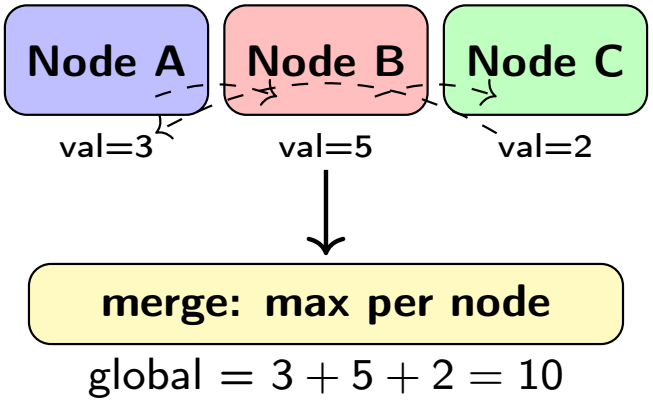
Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

Мотивация и интуиция

Почему CRDT?

- OT сложно реализовать корректно
- Семантическая реконсиляция требует координации
- Хотим: **локальные** операции + **гарантированная** сходимость



Ключевая интуиция:
Если структуры данных и операции спроектированы так, что результат *не зависит от порядка и количества применений*, конфликтов не возникает в принципе.

Демо

- The CRDT Dictionary: A Field Guide to Conflict-Free Replicated Data Types
- An introduction to Conflict-Free Replicated Data Types

Определение CRDT и SEC

CRDT — Conflict-free Replicated Data Type (Shapiro et al., 2011)

Реплицированный тип данных называется CRDT, если он реализует **сильную итоговую согласованность** (Strong Eventual Consistency, SEC).

SEC

Реплицированный объект удовлетворяет SEC, если:

1. **Eventual delivery:** каждое обновление, применённое на корректной реплике, в конечном счёте доставляется на все корректные реплики
2. **Strong Convergence:** корректные реплики, получившие *одинаковый набор* обновлений, находятся в **одинаковом состоянии**

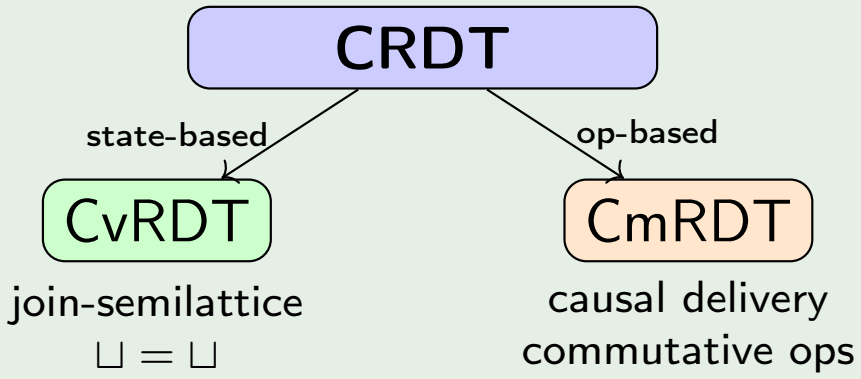
SEC vs. EC (Eventual Consistency)

EC гарантирует только eventual convergence «когда-нибудь»; SEC гарантирует convergence *сразу при доставке одинакового набора обновлений* — это значительно сильнее.

Strong Eventual Consistency

- При доставке одного и того же набора обновлений все реплики гарантированно равны
- Порядок доставки не важен
- Обычно достигается через CRDT:
  - операции **коммутативны**: $op_1 \circ op_2 = op_2 \circ op_1$
  - merge **ассоциативен, коммутативен и идемпотентен**: $(s_1 \sqcup s_2) \sqcup s_3 = s_1 \sqcup (s_2 \sqcup s_3)$

Таксономия (Shapiro et al., 2011)



Примеры: счетчики, множества, совместное редактирование документов.

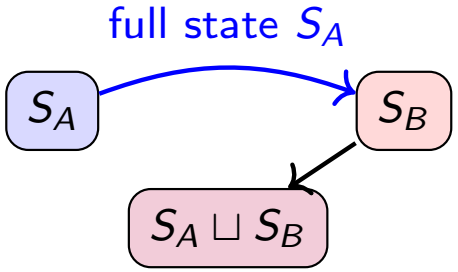
State-based vs. Operation-based CRDT

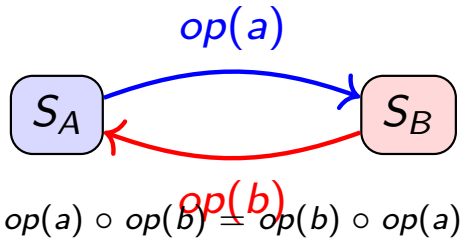
CvRDT (State-based)

- Передаём **всё состояние** реплике
- Merge через $\sqcup$ (join / LUB)
- Требование: join-semilattice
- + простота: любой канал, идемпотентность
- – затраты на передачу состояния

CmRDT (Op-based)

- Передаём **операции**
- Операции должны быть **коммутативны**
- Требование: *causal delivery* (FIFO или causal broadcast)
- + малый объём сообщений
- – надёжный канал с causal ordering





$op(a) \circ op(b) = op(b) \circ op(a)$

Эквивалентность

Определение CvRDT

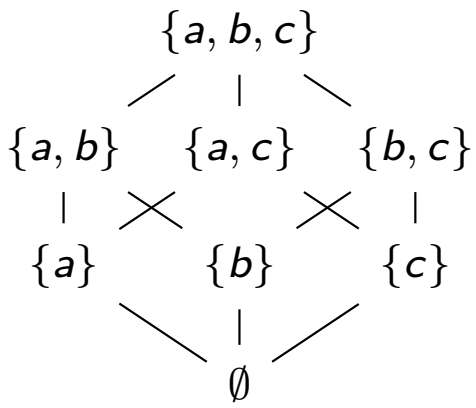
Join-Semilattice / CvRDT

$(S, \leq, \sqcup)$ — CvRDT, если:

- $(S, \leq)$ — частично упорядоченное множество
- $\forall a, b \in S$: $a \sqcup b = \sup\{a, b\}$ (least upper bound)
- $\sqcup$ коммутативна, ассоциативна, идемпотентна
- Каждое обновление **монотонно**: $s \leq \text{update}(s)$

Диаграмма Хассе для

$\mathcal{P}(\{a, b, c\})$:



Свойство монотонности:

Состояние реплики *только растёт* по решётке. Это гарантирует сходимость: после любого merge результат $\geq$ обоих операндов.

LUB явно:

$$s_1 \sqcup s_2 \geq s_1, \; s_1 \sqcup s_2 \geq s_2$$

Определение CmRDT

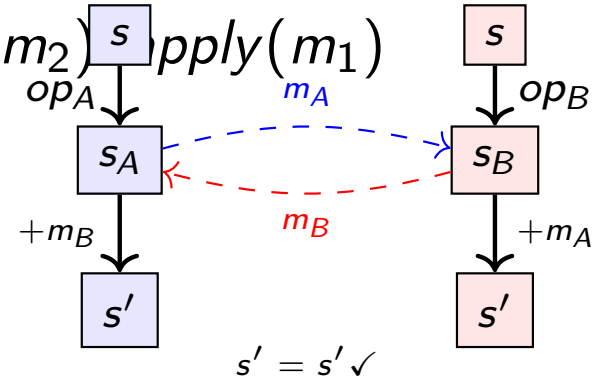
CmRDT — Op-based CRDT

Реплицированный объект $(S, do, apply)$ является CmRDT, если:

- $do(op, s)$ выполняется **локально** и генерирует сообщение m
- $apply(m, s)$ применяется на удалённых репликах
- Параллельные операции **коммутативны**:
  $apply(m_1, apply(m_2, s)) = apply(m_2, apply(m_1, s))$

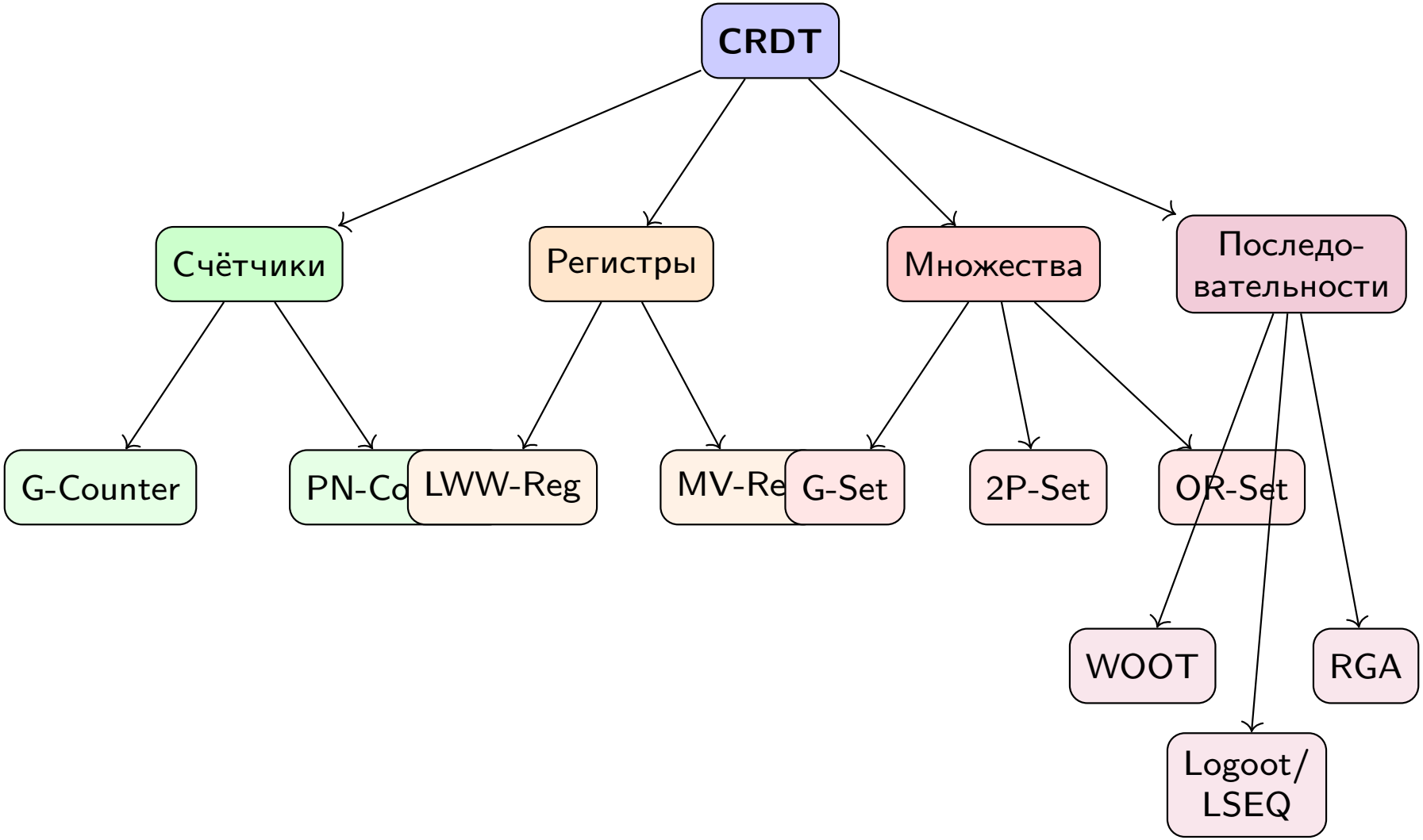
Условие коммутативности:

$$m_1 \parallel m_2 \Rightarrow apply(m_1) \circ apply(m_2) = apply(m_2) \circ apply(m_1)$$



Операции *конкурентны* ($m_1 \parallel m_2$), если ни одна не зависит от другой в частичном порядке событий (Lamport, 1978). **Доставка:** causal broadcast (например, по vector clock).

Зоопарк CRDT



Отечественные исследователи

Виктор Грищенко

- Автор **Chronofold** (2020) — CRDT для последовательностей на основе linked list с временными метками
- Концепция **Swarm** — децентрализованный объектная среда на основе CRDT
- Ранние работы по **RON** (Replicated Object Notation)

Юрий Сыровецкий

- Haskell-реализация CRDT (`crdts` на Hackage)
- Вклад в формализацию типов в функциональном стиле

Содержание

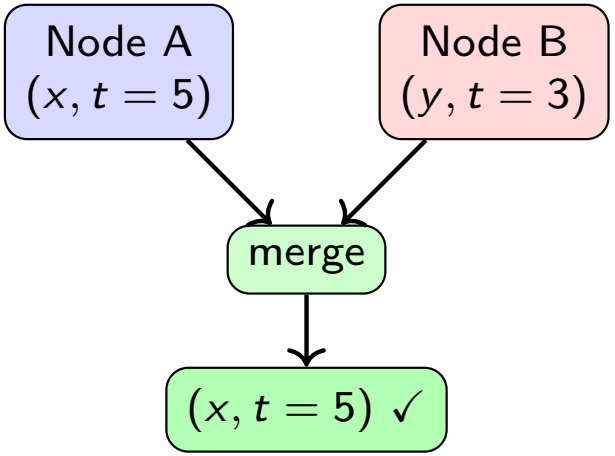
1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

Last-Writer-Wins Register (LWW-Register)

Определение:

- Состояние: (v, ts) — значение и временная метка
- `write`(v'): $(_, _) \to (v', now())$
- `read`: возвращает v
- `merge`$((v_1, t_1), (v_2, t_2))$:

$$= \begin{cases} (v_1, t_1) & \text{если } t_1 > t_2 \\ (v_2, t_2) & \text{иначе} \end{cases}$$



Метки: Lamport clock, physical clock (NTP), hybrid logical clock (HLC).

Применяется: Amazon DynamoDB (per-attribute LWW), Cassandra (по умолчанию).

Проблема LWW

Записи могут теряться! Если $t_A \approx t_B$, один `write` «перекрывает» другой — семантика last-write-wins.

Multi-Value Register (MVR)

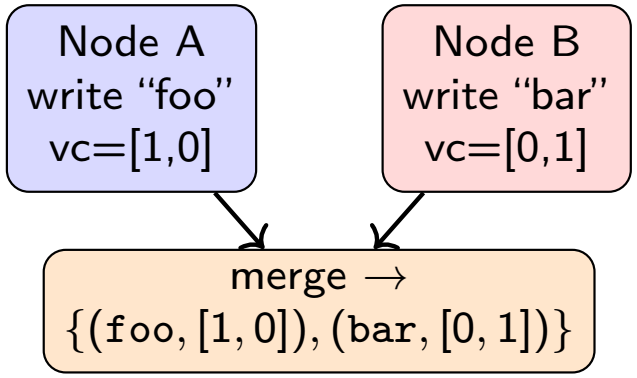
Идея:
Вместо одного значения хранить *множество* конкурентных значений.
Используем **vector clocks** для определения параллельности.

Состояние: множество пар $\{(v_i, \mathrm{vc}_i)\}$

write(v) на реплике i:
— инкрементируем $\mathrm{vc}[i]$
— удаляем все записи, покрытые новым VC
— добавляем (v, vc)

merge: объединяем множества, удаляем «dominated» записи.

read: если $|\mathrm{set}| > 1$ — конфликт; приложение решает



Concurrent writes!
Application picks or shows both.

Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

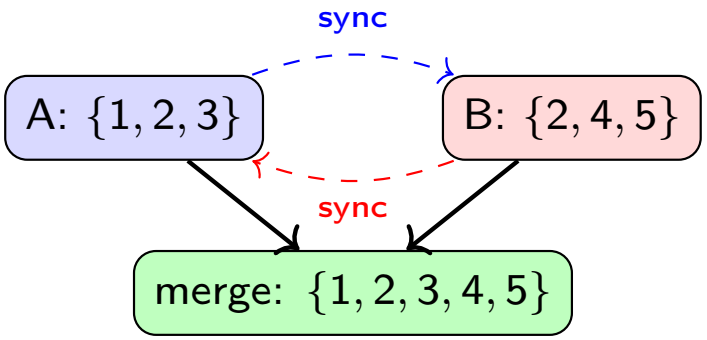
Grow-only Set (G-Set)

Простейший CRDT: элементы только добавляются, никогда не удаляются.
Формально:

- Состояние: $S \subseteq \mathcal{U}$
- $\texttt{add}(e)$: $S \leftarrow S \cup \{e\}$
- $\texttt{lookup}(e)$: $e \in S$
- $\texttt{merge}(S_1, S_2) = S_1 \cup S_2$

Решётка: $(\mathcal{P}(\mathcal{U}), \subseteq, \cup)$

Свойства:
$S_1 \cup S_2$ — LUB в решётке подмножеств. Монотонность: $S \subseteq S \cup \{e\}$. Все аксиомы CvRDT выполнены тривиально.



Применение

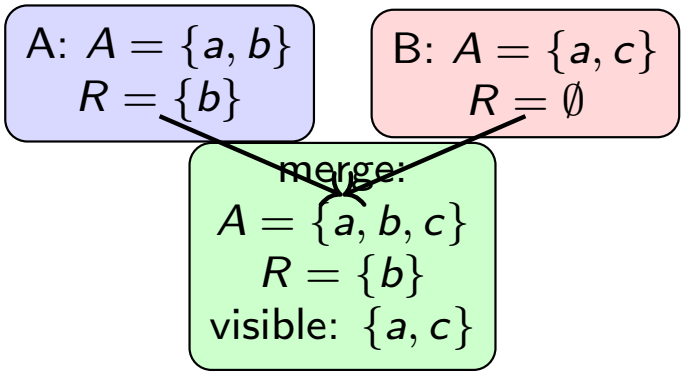
Набор тегов, список увиденных событий, распределённый blacklist, membership views.

Two-Phase Set (2P-Set)

Идея: разрешить *удаление*, но только **один раз** (deleted $\to$ нельзя добавить снова).
Состояние: два G-Set: A (добавленные), R (удалённые), $R \subseteq A$

$$\texttt{lookup}(e) \Leftrightarrow e \in A \setminus R$$

- $\texttt{add}(e)$: $A \leftarrow A \cup \{e\}$
- $\texttt{remove}(e)$: **только если** $e \in A$: $R \leftarrow R \cup \{e\}$
- $\texttt{merge}$: $\langle A_1 \cup A_2,\ R_1 \cup R_2 \rangle$



Ограничение: при конкурентных `add` и `remove` — побеждает `remove` (remove-wins bias) по умолчанию.

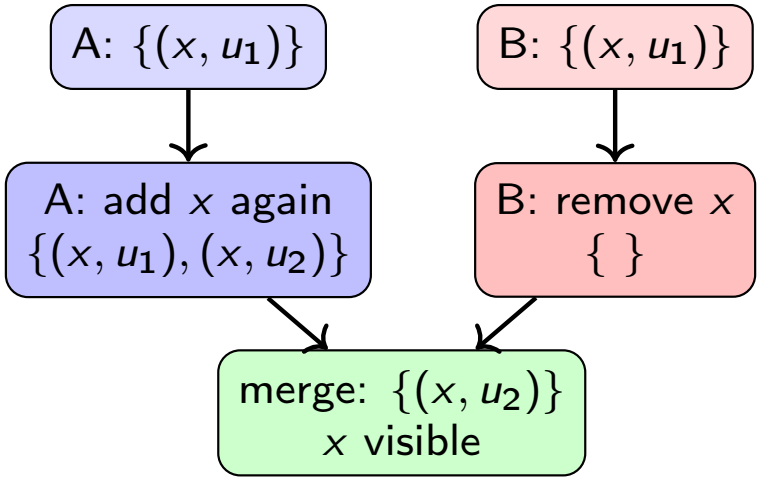
Observed-Remove Set (OR-Set)

Проблема 2P-Set: нельзя повторно добавить удалённый элемент.

OR-Set решает это с помощью уникальных тегов (Shapiro et al., 2011).

Состояние: $\{(e, \mathrm{uid})\}$ — мультимножество пар

- add(e): добавляем $(e, \mathrm{uid}_{\mathrm{fresh}})$
- remove(e): удаляем все пары $(e, _)$ *которые наблюдала текущая реплика*
- lookup(e): $\exists$ пара $(e, _)$ в состоянии
- merge: объединение множеств пар



add-wins: u_2 не был виден B

Positive-Negative Set и Unique Set

PN-Set (Positive-Negative)

- Два G-Set: P (добавления), N (удаления)
- `lookup`(e): $|P_e| > |N_e|$ (считаем вхождения)
- Позволяет повторное добавление после удаления
- Проблема: `remove` без предварительного add — некорректно

U-Set (Unique Set)

- Каждый элемент имеет **уникальный идентификатор**
- add(e, uid): добавить (e, uid)
- remove(e, uid): удалить (e, uid)
- Гарантирует: одна add $\rightarrow$ один `remove`
- Реализация: часто через OR-Set с UUID

Сравнение

Сравнительная таблица множеств

Тип	Удал.	Повтор. add	Conc. bias	Сложность
G-Set	нет	н/д	—	$O(n)$
2P-Set	да	нет	remove-wins	$O(n)$
OR-Set	да	да	add-wins	$O(n^2)$
PN-Set	да	да	зависит	$O(n)$

Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

Grow-only Counter (G-Counter)

Состояние: вектор $\mathrm{v} \in \mathbb{N}^n$ (по одному счётчику на узел)

$$\texttt{value}() = \sum_{i=1}^{n} \mathrm{v}[i]$$

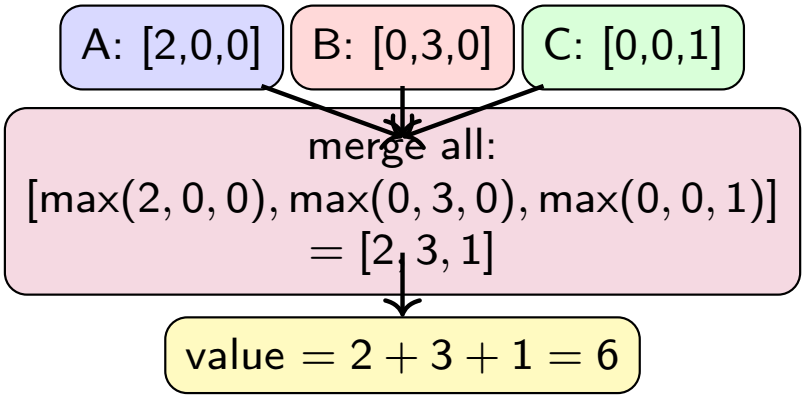
- `increment()` на узле i: $\mathrm{v}[i] \mathrel{+}= 1$
- $\texttt{merge}(\mathrm{v}_A, \mathrm{v}_B)[i] = \max(\mathrm{v}_A[i], \mathrm{v}_B[i])$

Решётка: $(\mathbb{N}^n, \leq_{\text{comp}}, \max)$
Где $\mathrm{u} \leq \mathrm{v} \Leftrightarrow \forall i : u_i \leq v_i$

Применение: счётчик лайков, просмотров, голосов.

Гарантия: никаких потерь инкрементов, любой порядок merge

3 узла, начало: [0,0,0]

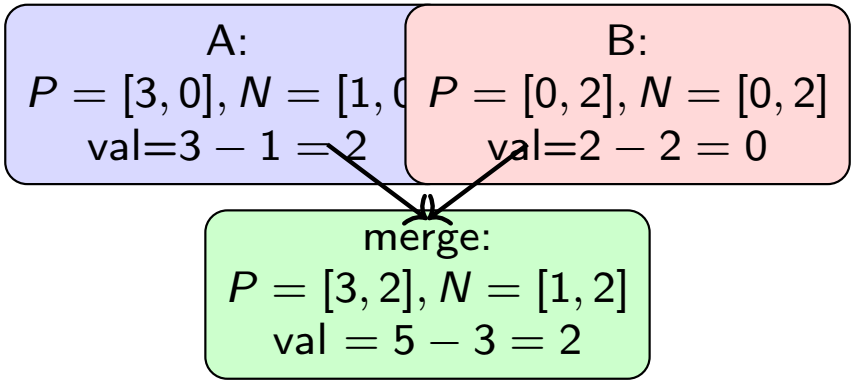


Positive-Negative Counter (PN-Counter)

Идея: объединить два G-Counter — P для increment, N для decrement.

$$\textbf{value}() = \sum_i P[i] - \sum_i N[i]$$

- `increment()` на узле i:
  $P[i] \mathrel{+}= 1$
- `decrement()` на узле i:
  $N[i] \mathrel{+}= 1$
- `merge`: max покоординатно для P и N



Ограничение: значение может быть отрицательным, но отдельные компоненты P, N только растут.

Применение: reddit score (upvotes - downvotes), баланс игровой

Открытые проблемы

Открытые вопросы

- Delta-CRDT (меньше трафика, Almeida et al., 2018)
- Pure op-based CRDT (Baquero et al., 2017)
- CRDT в БД: Electric SQL, Ditto, Automerge 2.0

Содержание

1. Оптимистическая репликация
2. История развития
3. CRDT
4. Регистры
5. Множества
6. Счётчики
7. Итоги

Итоги

- Нового домашнего задания 3 пока нет.